

Praktium 10 – Algoritma Struktur Data

Nama : Muhammad Yusuf Baihaqi Bawono

NIM : J0403251096

Praktikum 1 :

```
#=====
# Nama : Muhammad Yusuf Baihaqi Bawono
# NIM : J0403251096
# Kelas : TPL A1
#=====

def createGraph(V, edges):
    mat = [[0 for _ in range(V)] for _ in range(V)]
    # Menambahkan setiap edge ke adjacency matrix
    for it in edges:
        # Mengambil simpul awal dari edge
        u = it[0]
        # Mengambil simpul tujuan dari edge
        v = it[1]
        # Menandai bahwa ada hubungan dari u ke v
        mat[u][v] = 1
        # Karena graph tidak berarah (undirected),
        # maka v juga terhubung ke u
        mat[v][u] = 1

    # Mengembalikan adjacency matrix yang sudah dibuat
    return mat

if __name__ == "__main__":
    # Jumlah vertex
    V = 4

    # Daftar edge/hubungan antar simpul
    edges = [[0, 1], [0, 2], [1, 2], [2, 3]]

    # Membuat graph menggunakan fungsi createGraph
    mat = createGraph(V, edges)

    # Menampilkan judul output
    print("Adjacency Matrix Representation:")

    # Loop untuk menampilkan setiap baris matriks
    for i in range(V):
        # Loop untuk menampilkan setiap kolom matriks
        for j in range(V):
            # Menampilkan isi matriks
            print(mat[i][j], end=" ")

        # Pindah ke baris baru setelah satu baris selesai
        print()
```

```
> &
Adjacency Matrix Representation:
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
PS C:\Users\RPL\Documents\Code>
```

Praktikum 2 :

```
#=====
# Nama : Muhammad Yusuf Baihaqi Bawono
# NIM : J0403251096
# Kelas : TPL A1
#=====

def createGraph(V, edges):
    adj = [[] for _ in range(V)]

    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)

    return adj

if __name__ == "__main__":
    V = 4

    nodes = ["A", "B", "C", "D"]

    edges = [
        [0, 1],
        [0, 2],
        [1, 3],
        [2, 3]
    ]

    adj = createGraph(V, edges)

    print("Adjacency List Representation:\n")

    for i in range(V):
        print(f"{nodes[i]} : ", end="")

        for j in adj[i]:
            print(nodes[j], end=" ")

        print()
```

Adjacency List Representation:

```
A : B C
B : A D
C : A D
D : B C
```

Praktikum 3:

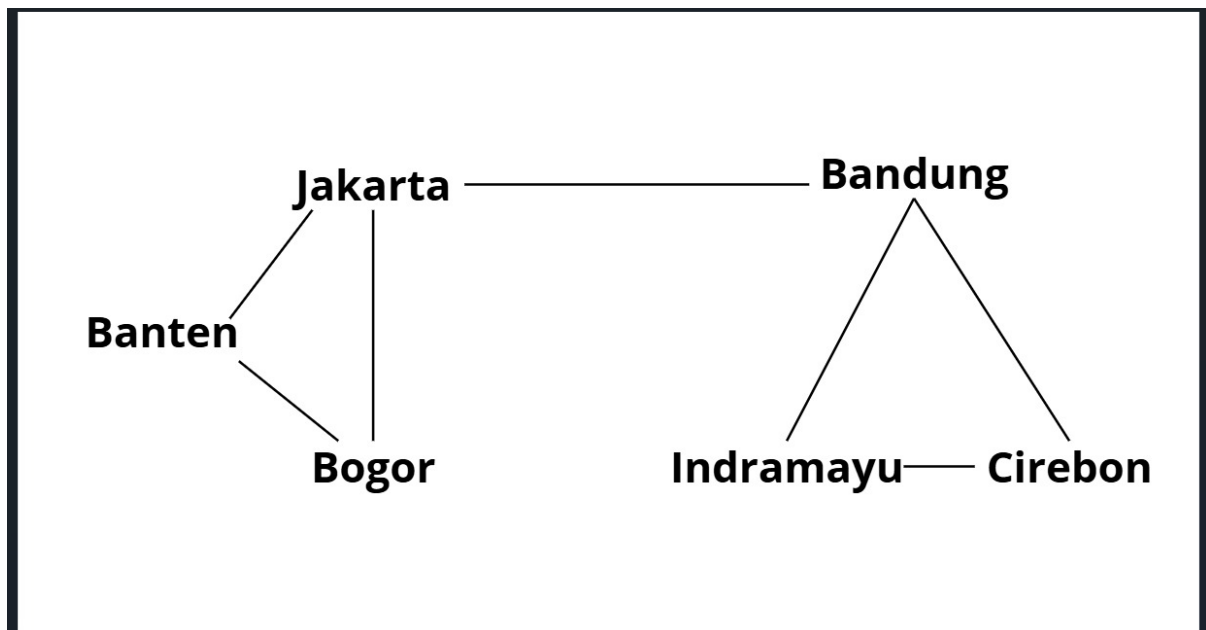
```
#####  
# Nama : Muhammad Yusuf Baihaqi Bawono  
# NIM : 30403251096  
# Kelas : TPL A1  
#####  
def createGraph(V, edges):  
  
    adj = [[] for _ in range(V)]  
  
    for it in edges:  
  
        u = it[0]  
        v = it[1]  
  
        adj[u].append(v)  
  
        adj[v].append(u)  
  
    return adj  
  
if __name__ == "__main__":  
  
    nodes = ['A', 'B', 'C', 'D']  
  
    V = 4  
  
    edges = [  
        [0, 1], # A - B  
        [0, 2], # A - C  
        [1, 2], # B - C  
        [2, 3] # C - D  
    ]  
  
    adj = createGraph(V, edges)  
  
    print("Adjacency List Representation:\n")  
  
    for i in range(V):  
  
        print(f"{nodes[i]} -> ", end="")  
  
        for j in adj[i]:  
  
            print(nodes[j], end=" ")  
  
        print()
```

Adjacency List Representation:

```
A -> B C  
B -> A C  
C -> A B D  
D -> C
```

Praktikum 4:

Graph :



```
=== ADJACENCY LIST ===
```

```
Jakarta -> Bandung, Banten, Bogor  
Bandung -> Jakarta, Indramayu, Cirebon  
Banten -> Jakarta, Bogor  
Bogor -> Jakarta, Banten  
Indramayu -> Bandung, Cirebon  
Cirebon -> Bandung, Indramayu
```

```
=== ADJACENCY MATRIX ===
```

	Jakarta	Bandung	Banten	Bogor	Indramayu	Cirebon
Jakarta	0	1	1	1	0	0
Bandung	1	0	0	0	1	1
Banten	1	0	0	1	0	0
Bogor	1	0	1	0	0	0
Indramayu	0	1	0	0	0	1
Cirebon	0	1	0	0	1	0

```
=== HUBUNGAN ANTAR NODE ===
```

```
Jakarta terhubung dengan Bandung  
Jakarta terhubung dengan Banten  
Jakarta terhubung dengan Bogor  
Bandung terhubung dengan Jakarta  
Bandung terhubung dengan Indramayu  
Bandung terhubung dengan Cirebon  
Banten terhubung dengan Jakarta  
Banten terhubung dengan Bogor  
Bogor terhubung dengan Jakarta  
Bogor terhubung dengan Banten  
Indramayu terhubung dengan Bandung  
Indramayu terhubung dengan Cirebon  
Cirebon terhubung dengan Bandung  
Cirebon terhubung dengan Indramayu
```

Code :

```

# =====
# Nama : Muhammad Yusuf Baihaqi Bawono
# NIM : J0403251096
# Kelas : TPL A1
# =====

# Adjacency List
graph = {
    "Jakarta": ["Bandung", "Banten", "Bogor"],
    "Bandung": ["Jakarta", "Indramayu", "Cirebon"],
    "Banten": ["Jakarta", "Bogor"],
    "Bogor": ["Jakarta", "Banten"],
    "Indramayu": ["Bandung", "Cirebon"],
    "Cirebon": ["Bandung", "Indramayu"]
}

# Nama node
nodes = [
    "Jakarta",
    "Bandung",
    "Banten",
    "Bogor",
    "Indramayu",
    "Cirebon"
]

# Adjacency Matrix
matrix = [
    #           JKT BDG BTN BGR IDM CRB
    [0, 1, 1, 1, 0, 0], # Jakarta
    [1, 0, 0, 0, 1, 1], # Bandung
    [1, 0, 0, 1, 0, 0], # Banten
    [1, 0, 1, 0, 0, 0], # Bogor
    [0, 1, 0, 0, 0, 1], # Indramayu
    [0, 1, 0, 0, 1, 0]  # Cirebon
]

# =====
# MENAMPILKAN ADJACENCY LIST
# =====

print("=== ADJACENCY LIST ===\n")

for node, neighbors in graph.items():
    print(f"{node} -> {' '.join(neighbors)}")

# =====
# MENAMPILKAN ADJACENCY MATRIX
# =====

print("\n=== ADJACENCY MATRIX ===\n")

# Header kolom
print(f"{'':12}", end="")
for node in nodes:
    print(f"{node:<12}", end="")
print()

# Isi matrix
for i in range(len(matrix)):
    print(f"{nodes[i]:<12}", end="")

    for j in range(len(matrix[i])):
        print(f"{matrix[i][j]:<12}", end="")

    print()

# =====
# MENAMPILKAN HUBUNGAN ANTAR NODE
# =====

print("\n=== HUBUNGAN ANTAR NODE ===\n")

for node, neighbors in graph.items():
    for neighbor in neighbors:
        print(f"{node} terhubung dengan {neighbor}")

```

Output :

```

=== ADJACENCY LIST ===

Jakarta -> Bandung, Banten, Bogor
Bandung -> Jakarta, Indramayu, Cirebon
Banten -> Jakarta, Bogor
Bogor -> Jakarta, Banten
Indramayu -> Bandung, Cirebon
Cirebon -> Bandung, Indramayu

=== ADJACENCY MATRIX ===

      Jakarta  Bandung  Banten  Bogor  Indramayu  Cirebon
Jakarta      0         1         1         1         0         0
Bandung      1         0         0         0         1         1
Banten       1         0         0         1         0         0
Bogor        1         0         1         0         0         0
Indramayu    0         1         0         0         0         1
Cirebon      0         1         0         0         1         0

=== HUBUNGAN ANTAR NODE ===

Jakarta terhubung dengan Bandung
Jakarta terhubung dengan Banten
Jakarta terhubung dengan Bogor
Bandung terhubung dengan Jakarta
Bandung terhubung dengan Indramayu
Bandung terhubung dengan Cirebon
Banten terhubung dengan Jakarta
Banten terhubung dengan Bogor
Bogor terhubung dengan Jakarta
Bogor terhubung dengan Banten
Indramayu terhubung dengan Bandung
Indramayu terhubung dengan Cirebon
Cirebon terhubung dengan Bandung
Cirebon terhubung dengan Indramayu

```

1. Jenis Graph

Grafik tersebut merupakan contoh dari Undirected Graph, saya menggunakan Undirected Graph karena cocok dengan penggunaannya dimana kita dapat berpergian 2 arah untuk setiap kota.

2. Alasan memilih edge

Edge digunakan untuk menunjukkan hubungan antar node atau kota.
Setiap edge menggambarkan adanya koneksi langsung antara dua kota.

Contoh:

Jakarta — Bandung

Bandung — Cirebon

Banten — Bogor

Jika terdapat edge, maka kedua node dianggap memiliki hubungan.

Alasan

Edge dipilih berdasarkan hubungan atau jalur yang ingin direpresentasikan pada graph. Pemilihan edge dilakukan agar:

setiap node memiliki koneksi,
graph terlihat saling terhubung,
memenuhi syarat minimal edge pada praktikum.

Graph yang saya gunakan memiliki 7 edge sehingga sudah memenuhi ketentuan minimal 6 edge.

3. Representasi mana lebih cocok

Adjacency list

Dikarenakan lebih mudah dibaca terutama dalam system peta kota

Alasan

Representasi adjacency list lebih cocok digunakan karena:

lebih hemat memori,

mudah dibaca,

hanya menyimpan node yang benar-benar terhubung,

Cocok untuk graph dengan sedikit edge.

Pada graph ini tidak semua node saling terhubung sehingga adjacency list menjadi lebih efisien.

4. Apakah Graph termasuk dense atau sparse

Graph ini termasuk **Sparse Graph**.

Penjelasan node dan edge :

1. Node (Vertex)

Pengertian Node

Node atau vertex adalah titik pada graph yang digunakan untuk merepresentasikan suatu objek, data, tempat, atau entitas tertentu. Node menjadi bagian utama dalam graph karena semua hubungan dimulai dari node.

Dalam kehidupan sehari-hari, node dapat diibaratkan sebagai:

- kota pada peta,
- orang dalam media sosial,
- komputer pada jaringan,
- stasiun pada jalur kereta,

- atau lokasi tertentu.
Node biasanya digambarkan dalam bentuk lingkaran atau titik.
-

Fungsi Node

Node memiliki beberapa fungsi utama, yaitu:

1. Menyimpan informasi atau data.
2. Menjadi titik penghubung dalam graph.
3. Merepresentasikan objek pada suatu sistem.
4. Membentuk struktur graph bersama edge.

2. Edge

Pengertian Edge

Edge adalah garis atau hubungan yang menghubungkan dua node dalam graph. Jika node merupakan objeknya, maka edge adalah hubungan antar objek tersebut.

Edge menunjukkan bahwa dua node memiliki koneksi atau relasi tertentu.

Dalam kehidupan sehari-hari edge dapat diibaratkan sebagai:

- jalan antar kota,
- pertemanan pada media sosial,
- kabel jaringan komputer,
- jalur kereta,
- atau hubungan antar data.

Edge biasanya digambarkan sebagai garis yang menghubungkan node.

Fungsi Edge

Fungsi edge antara lain:

1. Menghubungkan satu node dengan node lainnya.
2. Menunjukkan hubungan antar objek.
3. Membentuk jalur atau koneksi dalam graph.
4. Menentukan struktur hubungan antar data.

1. Adjacency List

Pengertian Adjacency List

Adjacency List adalah representasi graph yang menyimpan daftar node yang terhubung langsung dengan node tertentu. Pada metode ini, setiap node memiliki sebuah list atau daftar yang berisi node-node tetangga yang terhubung dengannya.

Dengan kata lain, adjacency list hanya menyimpan hubungan yang benar-benar ada pada graph.

Kelebihan Adjacency List

1. Hemat Memori

Adjacency list hanya menyimpan edge yang ada sehingga penggunaan memori menjadi lebih kecil.

Contoh:

Jika graph memiliki sedikit edge, adjacency list sangat efisien karena tidak menyimpan hubungan kosong.

2. Cocok untuk Sparse Graph

Sparse graph adalah graph yang jumlah edge-nya sedikit dibandingkan jumlah node. Adjacency list sangat cocok digunakan pada sparse graph karena hanya menyimpan hubungan yang diperlukan.

3. Mudah Melihat Tetangga Node

Jika ingin mengetahui node yang terhubung dengan suatu node, adjacency list lebih mudah dibaca.

2. Adjacency Matrix

Pengertian Adjacency Matrix

Adjacency Matrix adalah representasi graph menggunakan tabel atau matrix berbentuk baris dan kolom.

Baris dan kolom merepresentasikan node.

Isi matrix:

- 1 → terdapat hubungan,
- 0 → tidak terdapat hubungan.

Penjelasan

- Baris A kolom B bernilai 1 → A terhubung dengan B.
- Baris A kolom D bernilai 0 → A tidak terhubung dengan D.

Karena graph undirected, matrix bersifat simetris.

Cara Kerja Adjacency Matrix

Pada adjacency matrix:

1. Setiap node memiliki baris dan kolom.
 2. Program memeriksa hubungan menggunakan indeks matrix.
 3. Semua kemungkinan hubungan tetap disimpan, termasuk yang bernilai 0.
-

Kelebihan Adjacency Matrix

1. Pengecekan Hubungan Sangat Cepat

Untuk mengetahui apakah dua node terhubung cukup melihat nilai matrix.

2. Mudah Dipahami dalam Bentuk Tabel

Adjacency matrix terlihat rapi karena berbentuk tabel.

3. Cocok untuk Dense Graph

Dense graph adalah graph dengan jumlah edge yang sangat banyak.

Karena banyak node saling terhubung, adjacency matrix menjadi lebih efektif.

4. Akses Data Langsung

Menggunakan indeks array membuat pencarian hubungan menjadi cepat.

Kesimpulan

Adjacency List dan Adjacency Matrix merupakan dua cara untuk merepresentasikan graph. Adjacency List lebih hemat memori dan cocok digunakan pada sparse graph karena hanya menyimpan hubungan yang ada. Sedangkan Adjacency Matrix lebih cepat dalam pengecekan hubungan antar node dan cocok digunakan pada dense graph, tetapi membutuhkan memori lebih besar. Pemilihan representasi graph tergantung pada kebutuhan program dan jumlah hubungan pada graph.